TECNICATURA UNIVERSITARIA
EN PROGRAMACIÓN
A DISTANCIA



Metodología de Sistemas II

Trabajo Práctico
Entrega Continua y Resiliencia del Software

Farias, Gustavo
Matrícula: 101662
Comisión: M2025-13
Repositorio GitHub: [Lucenear/Tecnicatura_Programacion_UTN]

Parte 1: Integracion Continua (CI)

1- Filosofia de Trabajo:

Para mi, la filosofia Fail-Fast es clave porque me permite detectar errores apenas toco el codigo. Si algo falla, el pipeline se corta al instante. Esto me ahorra tiempo y recursos de servidores, y me da agilidad para arreglar el problema rapido mientras el contexto esta fresco en mi mente.

2- Anatomia del Pipeline (Stages):

- Lint & Static Analysis: Yo lo uso para revisar el estilo y errores basicos de sintaxis. Mitiga el riesgo de que entre codigo sucio o mal formateado al repositorio.
- Unit Tests: Aca verifico que las funciones individuales hagan lo que deben. Mitiga el riesgo de que un cambio rompa la logica interna del sistema.
- Security Scan: Busco vulnerabilidades conocidas en mis dependencias. Mitiga el riesgo de seguridad antes de que el codigo llegue a produccion.

3- Pipeline as Code:

Para mi, que el YAML sea el cerebro significa que toda la logica de integracion esta escrita como codigo. Lo guardo en el mismo repositorio porque asi tengo el versionado junto con mi aplicacion, y si alguien cambia el pipeline, queda registrado en el historial de commits.

Parte 2: Versionado Semantico (SemVer)

4- El Lenguaje del Cambio:

Partiendo de la v2.4.1, yo asignaria las siguientes versiones:

- Escenario A: Pasaria a la v2.4.2, porque es un parche menor que no cambia la interfaz.
- Escenario B: Pasaria a la v2.5.0, ya que agrego una funcionalidad nueva sin romper lo existente.
- Escenario C: Pasaria a la v3.0.0, porque es un cambio grande que obliga a otros sistemas a adaptarse.

5- Tags y Disparadores:

Yo uso los Tags de Git como marcas exactas en el historial. Tecnicamente, configuro mi pipeline para que, cada vez que detecta un nuevo Tag (como v1.0.0), se dispare automaticamente el proceso de compilacion y despliegue a produccion.

Parte 3: Entrega y Despliegue Continuo (CD)

6- Entrega vs. Despliegue:

La diferencia critica para mi es el boton final. En Entrega Continua, el codigo esta listo para salir, pero un humano decide cuando apretar el boton. En Despliegue Continuo, todo es automatico hasta produccion. Un caso donde prefiero Entrega Continua es en un sistema bancario, donde necesito coordinar el lanzamiento con mis compañeros del area legal antes de que los clientes lo vean.

7- La Gestion de Entornos:

Mi codigo pasa de Dev (donde desarrollo) a Staging (una copia exacta de produccion) y finalmente a Prod. El entorno de Staging me sirve para hacer pruebas finales de integracion y carga en un ambiente seguro, sin riesgo para los usuarios reales.

8- El Principio de Inmutabilidad:

Para mi, un Artefacto Inmutable es aquel que se compila una sola vez y no se vuelve a modificar. Es vital porque si recompilo el codigo al pasar de Staging a Prod, podria introducir diferencias o bugs nuevos. Al usar el mismo artefacto, tengo la certeza de que lo que probe es exactamente lo que llega al usuario.

Parte 4: Resiliencia y Mitigacion de Riesgos

9- El Mecanismo de Rollback:

- A nivel de infraestructura, un Rollback consiste en apagar los servidores con la version nueva y volver a encender los de la version anterior, o cambiar el trafico hacia atras.
- Haber trabajado con Tags de SemVer me ayuda porque ya tengo el artefacto exacto de la version anterior empaquetado y listo. Solo tengo que decirle al sistema que despliegue el Tag anterior, haciendo el proceso casi instantaneo.

10 Estrategias Progresivas:

En Blue-Green, yo tengo dos entornos completos y cambio todo el trafico de uno al otro de golpe. En Canary, libero la nueva version solo para un pequeño porcentaje de usuarios y voy aumentando el trafico de a poco.
La estrategia que me permite probar con un grupo reducido de usuarios reales antes del lanzamiento total es Canary Deployment.